ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO KỸ THUẬT CÁ NHÂN ĐỒ ÁN MÔN HỌC

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

TÌM HIỂU CÔNG NGHỆ NEXT.JS QUA ỨNG DỤNG TRA CỨU & KHÁM PHÁ PHIM ẢNH

CHUYÊN ĐỀ CÁ NHÂN: KIỂM TOÁN KIẾN TRÚC, XỬ LÝ LỖI HỆ THỐNG, SEO METADATA VÀ BỘ KIỂM THỬ TỰ ĐỘNG TRÊN NEXT.JS 16

GVHD: ThS. Phạm Thi Vương

Nhóm: 21

SVTH: Nguyễn Quốc Đại 3123560013

Vai trò: Kiểm toán kiến trúc, xử lý lỗi và kiểm thử

Thành phố Hồ Chí Minh, năm 2026

TÓM TẮT BÁO CÁO CÁ NHÂN (ABSTRACT)

Báo cáo trình bày đóng góp của sinh viên **Nguyễn Quốc Đại** (MSSV: 3123560013) trong đồ án MovieHub thuộc học phần *Các Công nghệ Lập trình Hiện đại*. Trọng tâm là kiểm toán kiến trúc, xử lý lỗi, hoàn thiện SEO và mở rộng kiểm thử trên Next.js App Router.

Các công việc chính gồm loại bỏ cơ chế ghi đè DNS và luồng TV chưa hoàn chỉnh; bổ sung trang lỗi tầng Root; xây dựng sitemap có liên kết tĩnh dự phòng, robots và ba security header; chuẩn hóa đầu vào tìm kiếm và bổ sung kiểm thử hồi quy. Phần thực hiện được phân biệt với boundary chi tiết phim, OpenGraph và các chức năng nền do nhóm xây dựng trước đó.

Bộ kiểm thử gồm **43 ca trong 11 tệp**, sử dụng công cụ có sẵn của Node.js. Công cụ minh chứng kết hợp TMDB mock với Next.js production để kiểm tra HTTP streaming, Data Cache và Server Action. Nội dung báo cáo được đối chiếu với mã nguồn, lịch sử thay đổi và các log đã lưu [6, 7]; kết quả mock không được suy rộng thành hiệu năng production hoặc độ phủ toàn bộ mã nguồn.

Từ khóa: *Next.js 16, App Router, Error Boundary, SEO Metadata, HTTP Security Headers, Node.js Test Runner.*

CHƯƠNG 1. TỔNG QUAN VÀ PHÂN CÔNG TRÁCH NHIỆM

1.1. Mục tiêu kỹ thuật

MovieHub lấy dữ liệu từ TMDB nên phải phân biệt kết quả rỗng hợp lệ, phim không tồn tại và lỗi dịch vụ. Mục tiêu cá nhân là xử lý đúng từng trạng thái, giữ ranh giới Server/Client và không đưa khóa API vào giao diện.

Các yêu cầu cụ thể gồm giao diện phục hồi theo phạm vi lỗi, sitemap còn hoạt động khi upstream thất bại, đầu vào được chuẩn hóa và kết quả kiểm tra có thể tái lập. Đây là các nội dung về trạng thái route, metadata và chất lượng phần mềm trong yêu cầu học phần.

1.2. Phạm vi cá nhân và phối hợp nhóm

Tác giả trực tiếp bổ sung bộ ba trang lỗi tầng Root, sitemap/robots, HTTP Security Headers, hồ sơ kiểm toán và các kiểm thử hồi quy; đồng thời tham gia sửa luồng lỗi TMDB, tìm kiếm và phân trang. Phân công chính được trình bày ở Bảng 1.1.

Bảng 1.1. Phạm vi công việc của các thành viên Nhóm 21

Thành viên	**Phạm vi chính**
Nguyễn Quốc Đại	Kiểm toán kiến trúc, xử lý lỗi tầng Root, SEO, cấu hình HTTP và mở rộng kiểm thử.
Nguyễn Công Quân	Kiến trúc App Router, tầng TMDB, trang chủ và các thành phần hiển thị phim.
Nguyễn Thái Vinh	Tương tác tìm kiếm, bộ lọc và chức năng danh sách xem sau.

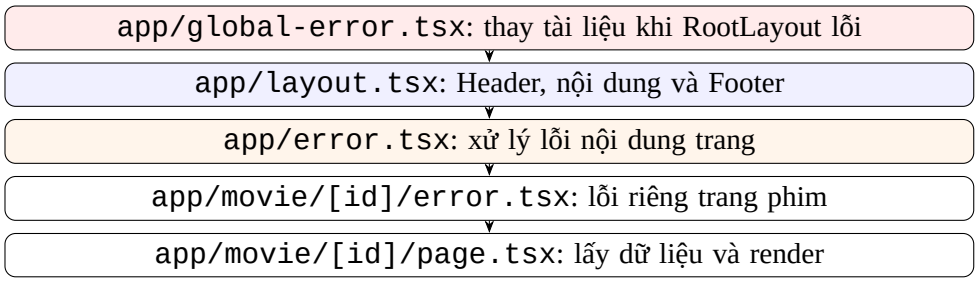
Boundary chi tiết phim và OpenGraph là phần nhóm đã xây dựng trước; báo cáo dùng chúng để giải thích hệ thống chung. Bộ công cụ minh chứng cũng được phát triển phối hợp, không quy toàn bộ mã nguồn hoặc bộ kiểm thử cho riêng một thành viên.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ THIẾT KẾ PHÒNG VỆ

2.1. Phạm vi Error Boundary trong App Router

Tệp `error.tsx` là Client Component, tạo giao diện dự phòng cho lỗi render trong phân đoạn và các thành phần con, nhưng không bọc layout cùng phân đoạn. Vì vậy, lỗi trang con có thể giữ Header/Footer; lỗi RootLayout cần `global-error.tsx` tự dựng tài liệu HTML thay thế [1].

Trong phiên bản Next.js đang dùng, `retry()` thử lấy lại dữ liệu và render cây con; `reset()` chỉ xóa trạng thái lỗi để render lại. Việc thử lại chỉ thành công khi nguyên nhân lỗi đã được khắc phục.



Hình 2.1. Phạm vi xử lý lỗi lồng nhau trong ứng dụng

2.2. Giao diện và phân biệt các tình huống 404

Giao diện lỗi dùng nền `#141414`, đỏ `#E50914` và chữ trắng theo thiết kế chung. Lỗi Server Component trong production được Next.js thay thông điệp chi tiết bằng thông điệp tổng quát; `error.digest` hỗ trợ đối chiếu log nếu có.

`notFound()` dành cho tài nguyên không tồn tại. Trước streaming, Next.js trả HTTP 404; sau khi streaming bắt đầu, phản hồi giữ HTTP 200. Cả hai trường hợp đều có `noindex` [2]. Lỗi dịch vụ 401, 500, 503 hoặc lỗi mạng phải được phân biệt với 404, tránh báo sai rằng phim đã bị xóa.

2.3. Metadata và an ninh HTTP

Metadata mô tả nội dung; sitemap cung cấp URL cho crawler; robots hướng dẫn phạm vi thu thập [3]. Các security header bổ sung hạn chế nhúng frame, MIME sniffing và thông tin referrer; chúng không thay thế kiểm tra đầu vào, xác thực hoặc bảo vệ bí mật [5].

CHƯƠNG 3. THIẾT KẾ BỘ KIỂM THỬ TỰ ĐỘNG

3.1. Công nghệ và phạm vi xác thực

Dự án khai báo Node.js từ 24, Next.js 16.4.5 và React 19.2.8. Bộ kiểm thử dùng `node:test`, `node:assert/strict` và cờ `--experimental-strip-types`, không cài thêm framework kiểm thử [4].

`scripts/test-resolver.mjs` xử lý alias `@/`, chuyển TSX bằng TypeScript và thay một số API Next.js bằng mock. React DOM kết xuất HTML để đối chiếu nội dung; cookie mock giữ trạng thái cho kiểm tra Server Action. Bảng 3.1 tổng hợp theo tệp thay cho liệt kê từng ca.

Bảng 3.1. Tổng hợp 43 ca kiểm thử trong thư mục tests/

Tệp kiểm thử	Ca	Nội dung xác thực chính
`api-search.test.mjs`	6	Query ngắn, cache header, giới hạn đầu vào, Unicode, tối đa 5 gợi ý và lỗi upstream.
`architecture-constraints.test.mjs`	2	Năm tiêu chí bộ lọc và cấm mẫu `as any` trong dịch vụ dùng chung.
`deduplicate.test.mjs`	1	Giữ phần tử đầu cùng ID, loại phim trùng và ID bằng 0.
`evidence-config.test.mjs`	1	TTL rút ngắn chỉ dùng cho HTTP loopback, trong khoảng 1–60 giây.
`metadata.test.mjs`	1	Hậu tố MovieHub chỉ xuất hiện một lần trong tiêu đề.
`page-flows.test.mjs`	14	Luồng search, browse, watchlist, sitemap; tham số lặp, phân trang và lỗi dịch vụ.
`r1-r2-features.test.mjs`	6	Bỏ mã chết, giới hạn tìm kiếm phim, robots, sitemap, header và cấu trúc trang lỗi.
`server-client-boundary.test.mjs`	1	Slider nhận thẻ đã render, không import trực tiếp MovieCard theo mẫu kiểm tra.
`server-fetch.test.mjs`	2	Request server, revalidate, timeout, mã HTTP và thông báo lỗi không chứa khóa.
`streaming-evidence.test.mjs`	1	Bộ kiểm chứng từ chối phản hồi đã đệm dù chia thành nhiều chunk.
`watchlist.test.mjs`	8	ID, cookie không tin cậy, giới hạn 200 phim và 3800 byte, thêm/xóa.
Tổng	**43**	**11 tệp; không phải tỷ lệ bao phủ mã nguồn.**

Kiểm tra ranh giới Server/Client chỉ dò mẫu default import `MovieCard` bằng biểu thức chính quy trong hai slider, không phân tích AST hay toàn bộ đồ thị module. Mock không kiểm chứng Data Cache thật; chuyển TSX không thay thế kiểm tra kiểu. Data Cache và kiểu dữ liệu được kiểm tra riêng bằng production runtime và TypeScript.

CHƯƠNG 4. HIỆN THỰC BẪY LỖI VÀ GIAO DIỆN 404

4.1. Global Error Boundary bảo vệ RootLayout

`app/global-error.tsx` khai báo `"use client"`, nhập CSS toàn cục và tự dựng `<html lang="vi">`, `<body>`. Khi RootLayout lỗi, giao diện này cung cấp thao tác phục hồi mà không phụ thuộc Header/Footer của layout.

Nút “Thử lại” gọi `retry()`, còn nút “Tải lại ứng dụng” thực hiện đoạn mã sau:

Listing 1. Xử lý tải lại tài liệu trong GlobalErrorBoundary

```
const handleReload = () => {
    window.location.reload();
};
```

Component ghi lỗi bằng `useEffect` và hiển thị `digest` khi có. Đây là log phía client, chưa phải hệ thống giám sát từ xa. Cả thử lại và tải lại đều có thể tiếp tục thất bại nếu nguyên nhân lỗi còn tồn tại.

4.2. Root Error Boundary bảo toàn khung ứng dụng

`app/error.tsx` nằm trong vùng nội dung của RootLayout, nên Header/Footer có thể tiếp tục hiển thị khi lỗi thuộc trang con. Giao diện có thông báo `role="alert"`, mã tham chiếu nếu có, nút thử lại và liên kết về trang chủ.

Trang phim có boundary riêng ở `app/movie/[id]/error.tsx`, được ưu tiên trước boundary tầng Root. Component này nhận cả `retry` và `reset`, ưu tiên `retry`; hai boundary tầng Root dùng `retry`. Boundary không tự bắt mọi lỗi event handler hay tác vụ bất đồng bộ ngoài luồng render.

4.3. Hai phạm vi giao diện không tìm thấy

`app/not-found.tsx` xử lý URL không khớp route; `app/movie/[id]/not-found.tsx` thông báo phim không tồn tại. Giao diện Root hiển thị số 404 lớn màu đỏ, biểu tượng dải phim bằng SVG và liên kết “Quay về Trang Chủ” tới `/`. Trang phim và `generateMetadata` kiểm tra ID bằng `/^[1-9]\d*$/` trước khi gọi dịch vụ; lỗi TMDB 404 được chuyển thành `notFound()`, còn lỗi khác tiếp tục được ném lên boundary.

Ở tầng dùng chung, `TMDBError` giữ mã HTTP, còn lỗi mạng hoặc timeout dùng `status = 0` và thông báo tổng quát. Tương tự, GET tìm kiếm trả HTTP 200 với danh sách rỗng hợp lệ, nhưng trả HTTP 500 khi dịch vụ thất bại. Cách phân loại này tránh biến sự cố upstream thành thông báo không có dữ liệu.

CHƯƠNG 5. HIỆN THỰC SEO METADATA VÀ AN NINH HTTP

5.1. Sitemap động với liên kết tĩnh dự phòng

`app/sitemap.ts` gọi `await connection()` để sinh dữ liệu khi có request; thao tác này không bỏ qua Data Cache. Dữ liệu trending dùng TTL 3600 giây, trừ cấu hình TTL rút ngắn dành cho mock loopback. Sitemap gồm trang chủ, `/browse` và URL của các phim trending, không phải toàn bộ kho TMDB. `/watchlist` được loại khỏi sitemap vì phụ thuộc cookie từng người dùng.

Khi lời gọi trending ném lỗi, nhánh `catch` ghi cảnh báo rồi `return staticRoutes`, giữ hai URL tĩnh. Riêng lỗi trang trending thứ hai được dịch vụ bỏ qua, nên vẫn có thể trả URL phim từ trang đầu. `lastModified` của phim dùng ngày phát hành hoặc ngày hiện tại nếu thiếu, chưa phản ánh chính xác thời điểm nội dung được cập nhật.

5.2. Robots và chính sách crawler

`app/robots.ts` cho phép thu thập `/`, loại trừ `/api/`, `/watchlist` và công bố sitemap. Base URL lấy từ `NEXT_PUBLIC_SITE_URL`, mặc định `https://moviehub.vn`; cần đặt đúng tên miền khi triển khai. Robots chỉ hướng dẫn crawler, không bảo vệ quyền truy cập.

RootLayout dùng mẫu tiêu đề `%s | MovieHub`; trang con trả tên nội dung chưa gắn hậu tố. Kiểm thử xác nhận ví dụ `Inception (2010) | MovieHub` không bị lặp hậu tố. OpenGraph trang phim ưu tiên poster, dự phòng backdrop hoặc mảng ảnh rỗng.

5.3. Security Headers và cấu hình cache

`next.config.ts` áp dụng ba header cho `/:path*`: `X-Frame-Options:DENY`, `X-Content-Type-Options:nosniff` và `Referrer-Policy:strict-origin-when-cross-origin`. Chúng lần lượt hạn chế nhúng frame, suy đoán MIME và thông tin referrer; cấu hình hiện chưa có CSP.

`cacheComponents: false` không tắt Data Cache của `fetch`; TTL vẫn theo từng dịch vụ. Build minh chứng dùng `.next-evidence` để tách khỏi build thường. Kiểm thử header mới xác nhận cấu hình, chưa chứng minh proxy/CDN giữ nguyên giá trị khi triển khai.

CHƯƠNG 6. KIỂM TOÁN KIẾN TRÚC VÀ ĐÓNG GÓP GIT

6.1. Đối soát 34 tiêu chí và sửa cơ chế DNS

Hồ sơ `baocao/DANH_GIA_REQUIRE.md` đối soát 34 tiêu chí giữa yêu cầu môn học, source, báo cáo và phản biện. Mục tiêu là tìm điểm lệch kiến trúc hoặc thiếu minh chứng; đây là hồ sơ tại thời điểm kiểm toán, không phải chứng nhận mọi tiêu chí của bản hiện tại.

Một vấn đề được xử lý là ghi đè `dns.lookup` trong tầng dịch vụ. Dù có kiểm tra hostname, việc thay hàm dùng chung vẫn tác động cấp tiến trình. Giải pháp là bỏ can thiệp toàn cục, dùng cơ chế phân giải của runtime; môi trường thử nghiệm trỏ `TMDB_BASE_URL` đến mock loopback. Việc này không đồng nghĩa dự án đã được kiểm chứng trên Edge Runtime.

6.2. Loại bỏ luồng phim bộ không hoàn chỉnh

Route `/tv` từng dùng thẻ dẫn đến `/movie/[id]`, khiến TV ID bị gửi sang movie endpoint. Tuyến `app/tv/page.tsx`, hàm `getTvPopular` trong `services/discover.ts` và liên kết điều hướng liên quan đã được loại bỏ; gợi ý tìm kiếm chuyển sang `/search/movie`. Phạm vi ứng dụng hiện là phim điện ảnh, tránh duy trì tính năng có luồng dữ liệu chưa đồng bộ.

6.3. Ma trận commit và Pull Request

Lịch sử Git được dùng để xác nhận phạm vi cá nhân. Bảng 6.1 chỉ giữ ba mốc tiêu biểu; phần thuyết minh tập trung vào kết quả kỹ thuật.

Bảng 6.1. Các mốc thay đổi tiêu biểu

Mốc	Nội dung chính	PR
`953db35`	Bỏ DNS patch và route TV; thêm trang lỗi Root, sitemap/robots, header và kiểm thử.	#8
`80fb08e`	Chuẩn hóa tham số, phân trang discover; thêm kiểm thử luồng trang, metadata và công cụ Chrome.	#10
`66ba616`	Sửa giới hạn Unicode, loại watchlist khỏi sitemap và cập nhật phụ thuộc.	#10

Các sửa đổi dùng chung còn gồm timeout 10 giây, thông báo lỗi mạng không chứa khóa API và `normalizeMovieParams`: chọn query đầu khi lặp, bỏ ký tự điều khiển, giới hạn 100 đơn vị UTF-16 và gọi `toWellFormed()` trước mã hóa URL. Bộ lọc được đối chiếu với danh sách giá trị hỗ trợ.

CHƯƠNG 7. THỰC NGHIỆM VÀ GIỚI HẠN MINH CHỨNG

7.1. Kết quả kiểm thử được lưu trong kho mã nguồn

`baocao/evidence/test-output.txt` ghi nhận ngày 06/11/2026: 43 ca đạt, không thất bại hoặc bỏ qua, exit code 0. Log lint, kiểm tra kiểu và build thuộc lượt ngày 05/11/2026 dùng Next.js 16.3.6, trong khi phiên bản hiện khai báo là 16.4.5. Vì khác trạng thái source, các kết quả này được trình bày như minh chứng lịch sử, không xác nhận toàn bộ bản hiện tại.

Với watchlist, ngưỡng 3800 byte tính trên tên `mh_watchlist`, dấu = và JSON đã qua `encodeURIComponent`, không gồm thuộc tính Set-Cookie. Kiểm thử dùng 199 ID dài: thêm ID tiếp theo bị từ chối do dung lượng dù chưa vượt 200 phim, cookie cũ vẫn giữ nguyên. Xóa phim vẫn được phép khi danh sách đủ 200 mục.

Lượt production mock lưu `status=passed`, dùng HTTP loopback và TTL 15 giây. Script tự chọn cổng trống; cổng 4100 chỉ là mặc định khi chạy mock thủ công. Các kết quả chính nằm trong Bảng 7.1.

Bảng 7.1. Kết quả và phạm vi minh chứng đã lưu

Minh chứng	Kết quả ghi nhận
Kiểm thử tự động	`test-output.txt`: 43/43 ca; chỉ xác nhận các tình huống đã kiểm tra.
Chất lượng mã và build	`lint-output.txt`, `typecheck-output.txt`, `build-output.txt`: exit code 0; bảng build liệt kê 9 route.
HTTP route	`routes-http.json`: HTML có bốn hàng phim, không có khóa API trong HTML đã kiểm tra; query lặp và ID sai được xử lý.
HTTP streaming	`streaming-mock.json`: nội dung chính/fallback đến ở 63ms, diễn viên ở 2564ms, phim tương tự ở 4064ms.
Data Cache	`cache-mock.json`: bốn endpoint có chuỗi đếm 0, 1, 1, 2, 2; nội dung đổi sau refresh.
Server Action	`watchlist-http.json`: thêm/xóa được giữ qua GET mới; phiên cookie riêng không có phim đã lưu.

Các tên tệp trong bảng thuộc `baocao/evidence/`.

Mock chủ động trì hoãn diễn viên 2,5 giây và phim tương tự 4 giây; các mốc trên là thời điểm nhận HTML, không phải chỉ số vẽ giao diện. Bộ đếm cache lần lượt biểu diễn trước request, miss, hit, refresh sau hết hạn và hit sau refresh. Đây là Data Cache cục bộ, không phải cache trình duyệt hoặc CDN.

7.2. Tái lập và đọc kết quả đúng phạm vi

Quy trình chạy lại từ thư mục gốc:

1. Dùng Node.js từ 24, cài phụ thuộc bằng `npm ci`; ghi lại phiên bản và trạng thái source. Kiểm chứng trình duyệt cần Chrome/Chromium hoặc `CHROME_BIN` hợp lệ.
2. Chạy `npm test`, `npm run lint`, `npx tsc --noEmit`; đọc tổng kết và exit code của từng lệnh.
3. Chạy `npm run evidence` để build production, khởi động mock và kiểm tra HTTP cùng tương tác Chrome. Có thể đặt `MOVIEHUB_EVIDENCE_DIR` để giữ riêng lượt mới.
4. Chấp nhận lượt mới khi `mock-runtime.json` có `status=passed`, các artifact cùng `run_id` và `source_sha256`. Thử riêng build và TMDB thật khi có khóa, cấu hình và kết nối phù hợp.

Script hiện có kiểm chứng Chrome cho tìm kiếm, bộ lọc, lịch sử điều hướng, hoàn tác watchlist và focus dialog; thư mục minh chứng đã lưu chưa có `browser-interactions.json`. Các ảnh watchlist ngày 23/10/2026 thuộc lượt riêng, không xác nhận toàn bộ bản sửa sau đó.

Kiểm thử boundary hiện chủ yếu xác nhận cấu trúc, chưa có kịch bản gây lỗi RootLayout rồi kiểm tra phục hồi. Các số đo mock không chứng minh Core Web Vitals, hiệu năng TMDB thật hoặc toàn bộ khả năng truy cập.

CHƯƠNG 8. KẾT LUẬN

8.1. Đánh giá kết quả đạt được

Đóng góp cá nhân tập trung vào tính đúng đắn và khả năng kiểm chứng của ứng dụng:

- Kiểm toán yêu cầu, bỏ DNS monkey-patch, luồng TV chưa hoàn chỉnh và mã không dùng.
- Bổ sung xử lý lỗi tầng Root, phân biệt tài nguyên không tồn tại với lỗi dịch vụ và cung cấp thao tác thử lại.
- Hoàn thiện sitemap, robots, ba security header và kiểm tra metadata tiêu đề.
- Mở rộng kiểm thử lên 43 ca trong 11 tệp, cùng nhóm duy trì công cụ minh chứng production với TMDB mock.

Các thay đổi giúp giảm tác động ngoài phạm vi, tránh thông báo sai về dữ liệu và tạo cơ sở kiểm tra hồi quy khi tiếp tục phát triển.

8.2. Hạn chế và định hướng phát triển

Ưu tiên tiếp theo là thu lại toàn bộ minh chứng từ cùng trạng thái source, bổ sung kịch bản phục hồi boundary trên trình duyệt và dùng thời điểm cập nhật nội dung phù hợp cho sitemap. Khi triển khai production, cần đánh giá CSP, đo Core Web Vitals và bổ sung giám sát lỗi tập trung.

TÀI LIỆU THAM KHẢO

[1] Vercel, “Next.js: error.js.” https://nextjs.org/docs/app/api-reference/file-conventions/error.

[2] Vercel, “Next.js: not-found.js.” https://nextjs.org/docs/app/api-reference/file-conventions/not-found.

[3] Vercel, “Next.js: Metadata Files.” https://nextjs.org/docs/app/api-reference/file-conventions/metadata.

[4] Node.js, “Test runner; Modules: TypeScript.” https://nodejs.org/api/test.html; https://nodejs.org/api/typescript.html.

[5] OWASP Foundation, “Secure Headers Project.” https://owasp.org/www-project-secure-headers/.

[6] Nhóm 21, “MovieHub: mã nguồn và lịch sử phát triển.” https://github.com/congquan92/ModernProgrammingTechnologies.

[7] Nhóm 21, “Hồ sơ minh chứng MovieHub,” `baocao/evidence/` và `baocao/HUONG_DAN_THU_MINH_CHUNG.md`.